

Guess Market - Exercise 1

Submitted By

- **Name:** Nave Kluska
- **ID:** 324168160
- **Email:** navekluska@gmail.com

GitHub Repository

<https://github.com/NaveKluska/GuessMarket>

Bonuses Implemented

- **Bonus 1:** System State Save and Load (שמירה וטעינה של המע) - Implemented via Options 6 and 7 in the main menu.

User Manual (How to Use)

The application is driven by an 8-option main menu. To interact with the system:

- Simply enter the **index number** (1-8) of the action you wish to perform and press Enter.
- When selecting items from a list (e.g., choosing an event or an option to buy shares in), always enter the corresponding **index**

- number** shown on the screen, not the item's name.
- When prompted for a file (such as loading an XML file or saving/loading the system state), provide the **full absolute path** to the file (e.g., `C:\path\to\file.xml`).
- If you provide invalid input, the system will display an error and return you safely to the main menu.

General System Overview & Design Choices

This project implements the core engine and a console-based user interface for the Guess Market system, featuring an LMSR (Logarithmic Market Scoring Rule) trading mechanism.

Design Choices & Assumptions:

- Menu Interaction:** If a user encounters an error during a menu operation (e.g., providing an invalid file path, inputting an incorrect option index), the system catches the error, displays a cohesive and clear error message, and safely returns the user to the main menu. This ensures the application remains robust, non-blocking, and prevents the user from ever getting "stuck."
- Account Balance (MM Account):** The engine tracks the Market Maker's net pocket via `accountBalance`. It initializes at `0.0`. When users buy shares, the cost + commission is deposited. Upon closing the event, the net payouts are deducted. A negative balance accurately reflects the MM's subsidization loss, fully complying with the exercise edge cases.
- DTO Isolation:** To protect the integrity of the engine's internal state, the system employs strict Data Transfer Objects (DTOs). All lists exposed to the UI are copied into new standard `java.util.List` objects. This guarantees that the UI receives a pristine snapshot of the data and physically cannot tamper with the engine's internal state.

Core Classes Documentation

1. Program & GuessMarketConsoleAppEX1 (Application Bootstrapping)

The entry point of the system. Rather than burying initialization logic in the UI, the application is elegantly bootstrapped in

`GuessMarketConsoleAppEX1`. Here, the dependencies (such as the JAXB XML parser, the commission calculator, and the core Engine) are instantiated and cleanly injected into one another. `Program` merely contains the `main` method that creates an instance of `GuessMarketConsoleAppEX1` and runs it.

2. MarketEngineImpl (Core Engine)

The central manager of the entire application. It acts as the mediator between the UI and the underlying data models. It manages all loaded events, orchestrates the purchase of shares, calculates payouts upon event closure, and interacts with the XML parser to load initial system data.

3. LmsrEvent & Event (Data Models)

- **Event (Abstract):** The base class representing a market event. It tracks the event's options, commission settings, transaction history, and the MM's account balance.
- **LmsrEvent:** Extends `Event` to specifically implement the Logarithmic Market Scoring Rule. It provides the exact mathematical formulas for calculating the marginal price (probability) of an option and the cost of purchasing shares based on the liquidity parameter b_i .

4. EX1_JAXB_XMLFileParser (XML Parsing)

Responsible for unmarshalling the provided XML file into Java objects using JAXB. It performs brief but necessary application-level validation on the data (ensuring exactly two options, valid commission bounds, etc.) before safely passing the data to the Engine.

5. `StandardCommissionCalculator` (Billing)

A dedicated utility class that calculates the commission for a transaction. It correctly segregates the `ON_PURCHASE` and `ON_CLOSE` commission phases, ensuring that fees are only applied at the appropriate times according to the event's specific configuration.

6. `ConsoleUIEX1` (User Interface)

The active console module that interacts with the user. It presents the menu, gathers and validates raw user inputs, calls the appropriate methods on the `MarketEngine`, and formats the returned DTOs into a clean, readable text layout.

7. Data Transfer Objects (DTOs)

The `guessmarket.dto` package contains completely isolated, immutable POJOs (`EventSummaryDTO`, `EventDetailsDTO`, `OptionDTO`, `TransactionDTO`, `ReceiptDTO`). These objects are responsible for safely transferring data from the Engine to the Console UI without ever exposing internal references.